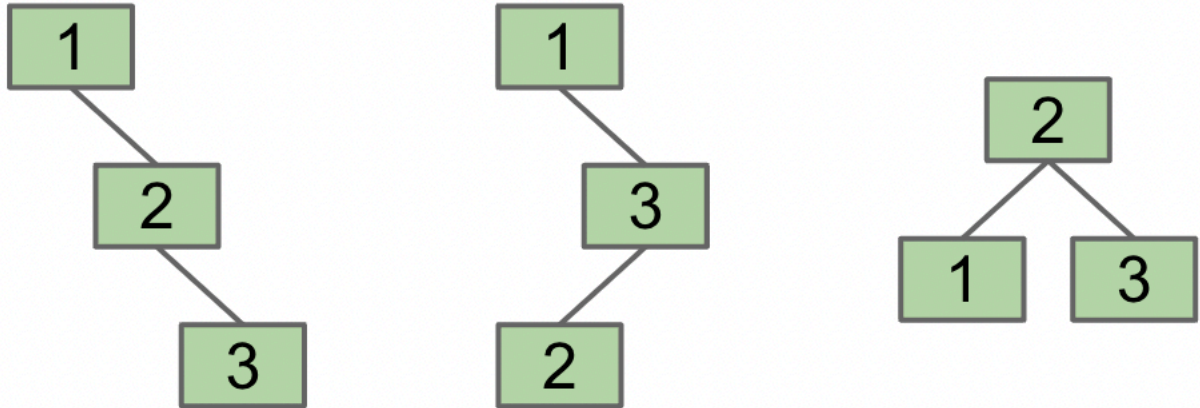


17.1 Rotating Trees

Wonderfully balanced as they are, B-Trees are really difficult to implement. We need to keep track of the different nodes and the splitting process is pretty complicated. As computer scientists who appreciate clean code and a good challenge, let's find another way to create a balanced tree.

Multiple Structures of BST

For any BST, there are multiple ways to structure it so that you maintain the BST invariants. Earlier, we talked about how **inserting** elements in different orders will result in a different BST. The sequence in which one inserts into a BST will affect its structure. The BSTs below all consist of the elements 1, 2, and 3, yet all have different structures.



► Exercise: For each tree shown above, provide an order of insertion that yields the structure.

Tree Rotation

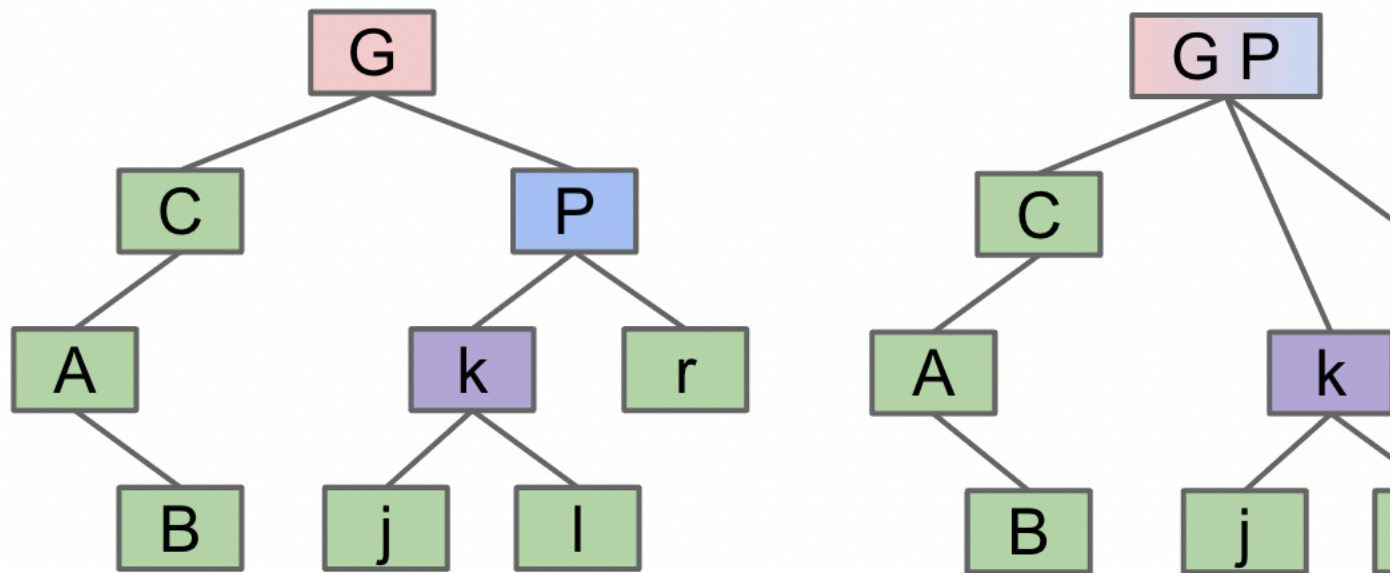
The formal definition of rotation is:

`rotateLeft(G)`: Let x be the right child of G . Make G the new left child of x .

`rotateRight(G)`: Let x be the left child of G . Make G the new right child of x .

We will slowly demystify this process in the next few paragraphs.

Below is a graphical description of what happens in a left rotation on the node G :



```
rotateLeft(G)
```

The written description of what happened above is this:

- G's right child, P, merges with G, bringing its children along.
- P then passes its left child to G and G goes down to the left to become P's left child.

You can see that the structure of the tree changes as well as its height. We can also rotate on a non-root node. We just disconnect the node from the parent temporarily, rotate the subtree at the node, then reconnect the new root.

Implementation

Here are the implementations of `rotateRight` and `rotateLeft`:

```
private Node rotateRight(Node h) {
    // assert (h != null) && isRed(h.left);
    Node x = h.left;
    h.left = x.right;
    x.right = h;
    return x;
}

// make a right-leaning link lean to the left
private Node rotateLeft(Node h) {
    // assert (h != null) && isRed(h.right);
    Node x = h.right;
    h.right = x.left;
    x.left = h;
    return x;
}
```

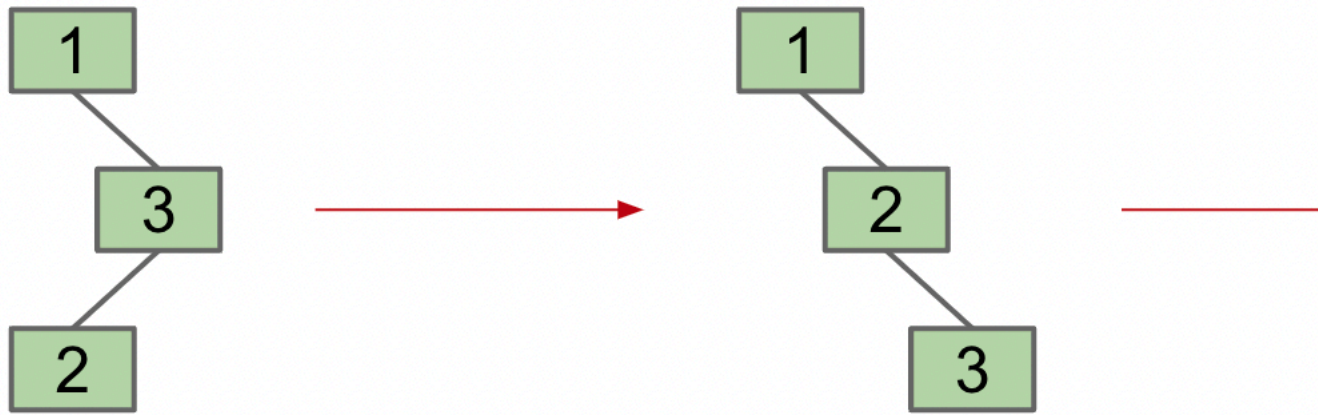
You may be wondering how does the parent node know about which node to point to after we rotate? That's why we are returning `x`, and other parts of our code will make use of this information to correctly update the parent node's pointer.

Balancing BSTs with Tree Rotation

```
{% embed url="https://www.youtube.com/watch?index=2&list=PL8FaHk7qbOD6aKgTz2W-foDiTeBEaBoS3&v=b4-2-6R2gzU" %}
```

With rotations, we can actually balance a tree.

Consider the example below:



We can do balance the given BST on the left by calling:

1. rotateRight(3)
2. rotateLeft(1)

The main observation to make for tree rotation is that it is possible to **shorten** or **lengthen** a tree, while maintaining the search tree's property.

For more examples, see the demo in [these slides](#).

17.2 Creating LLRB Trees

We said in the previous section that we really like 2-3 trees because they always remain balanced, but they are very hard to implement. On the other hand, BSTs can be unbalanced, but we like how simple and intuitive they are. Is there a way to combine the best of two worlds? Why not create a tree that is implemented using a BST, but is structurally identical to a 2-3 tree and thus stays balanced?

(Note that in this chapter we will be honing in on 2-3 Trees specifically, not 2-3-4 trees)

Entering Red Black Trees

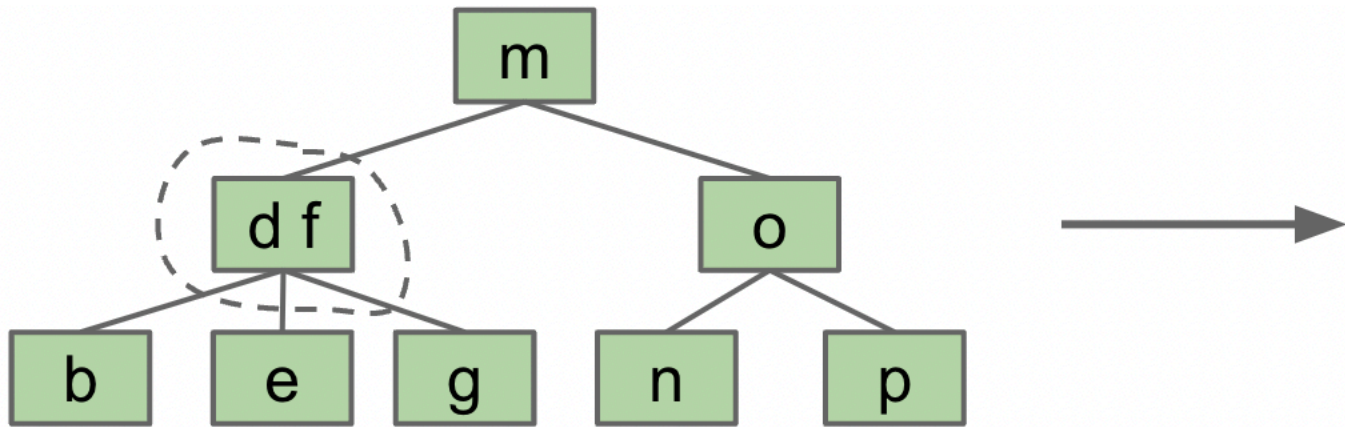
From 2-3 Tree to What?

We are going to create this tree by looking at a 2-3 tree and asking ourselves what kind of modifications we can make in order to convert it into a BST.

For a 2-3 tree that only has 2-nodes (nodes with 2 children), we already have a BST, so we don't need to make any modifications!

However, what happens when we get a 3-node?

One thing we could do is create a "glue" node that doesn't hold any information and only serves to show that its 2 children are actually a part of one node.

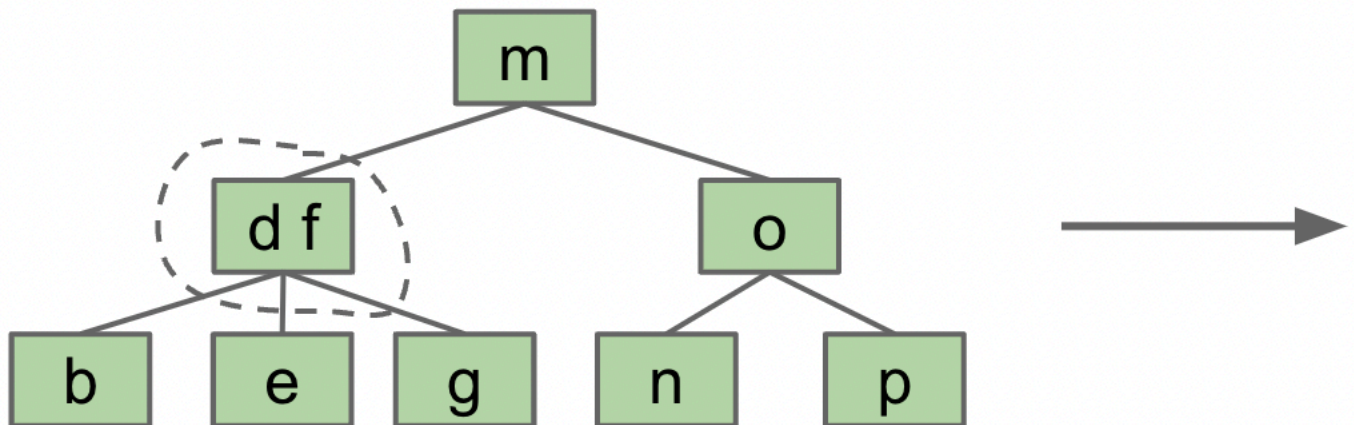


Naive Solution: Dummy “glue” node

However, this is not an elegant solution because we are taking up more space and the code will be ugly. So, instead of using glue nodes we will use **glue links** instead!

To LLRB Tree

To transform dummy glue nodes to glue links, we choose arbitrarily to make the left element a child of the right one. This results in a **left-leaning** tree.



Left Leaning Red-Black Tree

We show that a link is a glue link by making it red. Normal links are black. Because of this, we call these structures **left-leaning red-black trees (LLRB)**. We will be using left-leaning trees in this course.

One-to-One Correspondence:

Left-Leaning Red-Black trees have a **1-1 correspondence with 2-3 trees**. Every 2-3 tree has a **unique** LLRB red-black tree associated with it.

As for 2-3-4 trees, they maintain correspondence with standard Red-Black trees.

Properties of LLRBs

{% embed url="https://www.youtube.com/watch?index=4&list=PL8FaHk7qbOD6aKgTz2W-foDiTeBEaBoS3&v=4nZFgj7t52E" %}

Below is a summary of the properties/invariants of LLRB Trees:

- 1-1 correspondence with 2-3 trees.
- No node has 2 red links.
- There are no red right-links.
- Every path from root to leaf has the same number of black links (because 2-3 trees have the same number of links to every leaf).
- Height is no more than $2 \times \text{height} + 1$ of the corresponding 2-3 tree.
- The height of a red-black tree is proportional to the log of the number of entries.

17.3 Inserting LLRB Trees

We can always insert into an LLRB tree by inserting into a 2-3 tree and converting it as we've done before. However, this would be contrary to our original purpose of creating LLRBs, which was to avoid the complicated implementation of a 2-3 tree!

Instead, we **insert into the LLRB as we would with a normal BST**. However, this could break its 1-1 mapping to a 2-3 tree, so we will use **rotations** to massage the tree back into a proper structure.

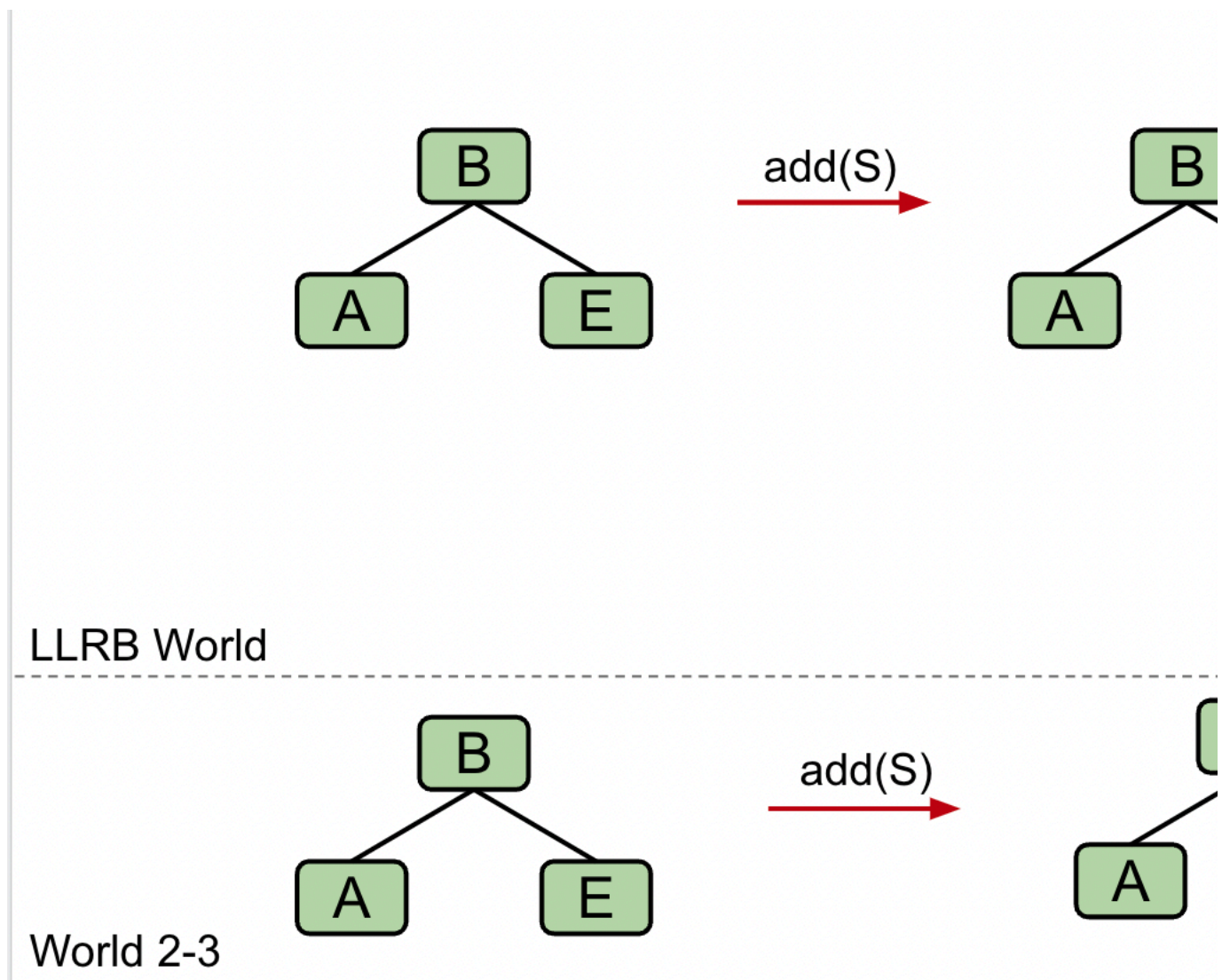
Maintaining Proper Structure

When inserting into an LLRB tree, we always insert the new node with a **red link** to its parent node. This is because in a 2-3 tree, we are always inserting by adding to a leaf node, the color of the link we add should always be red.

But sometimes, inserting red links at certain places might lead to cases where we break one of the invariants of LLRBs.

Below are three such cases where we need to perform certain tasks address in order to maintain the LLRB tree's proper structure:

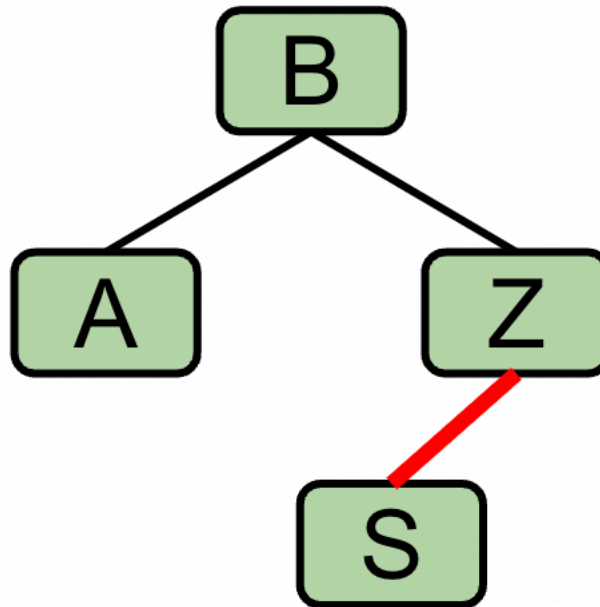
- Case 1: Insertion results in a right leaning "3-node -> Left Leaning Violation"
 - **Task 1: rotateLeft(node)**
 - Recall, we are using *left-leaning* red black trees, which means we can never have a right red link.
 - To address this, we will need to use a **rotation** in order to maintain the LLRB invariant: "There are no right red links".
 - If the left child is *also* a red link, then we will temporarily allow it for purposes that will become clearer in case 2.



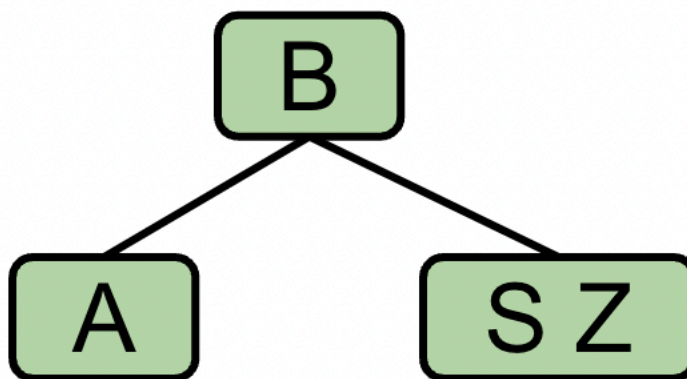
Case 1: Inserting a Right Red Link

- Case 2: Double Insertion on the Left -> Incorrect 4 Node Violation

- Task 2: rotateRight(node)
- If there are 2 left red links, then we have a 4-node which is illegal.
- To address this, we will first **rotate** to create the same tree seen in case 1 above. Then, in both situations, we will address the temporary issue of having an illegal 4-node in case 3.



LLRB World

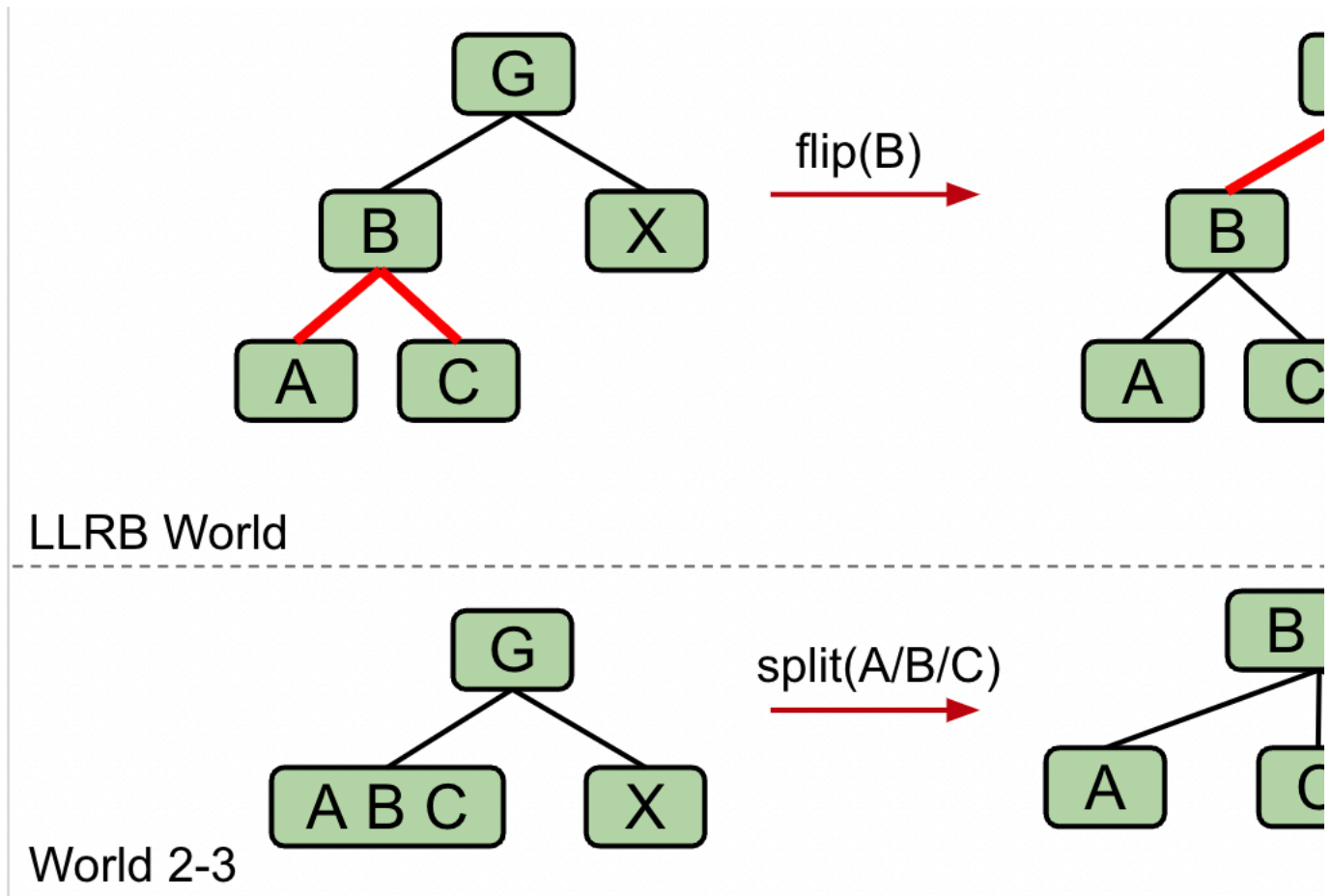


World 2-3

Case 2: Double Insertion on the Left

- Case 3: Node has Two Red Children -> Temporary 4 Node

- Task 3: Color Flip
- Finally, to address the issue of having illegal 4-node by **flipping the colors** of all edges touching "S" above.
- This is equivalent to pushing up the middle node in a 2-3 tree.



Case 3: Two Red Links

Note: You may need to go through a series of rotations in order to complete the transformation. It's possible that one operation will result in additional LLRB rule violations, which we can fix with the corresponding operation.

The process is: while the LLRB tree does not satisfy the 1-1 correspondence with a 2-3 tree or breaks the LLRB invariants, perform task 1, 2, or 3 depending on the condition of the tree until you get a legal LLRB.

Summary of Maintenance Operations

- When inserting: Use a **red link**. Insert in the same way as inserting into a BST.
- If there is a right-leaning “3-node”, we have a **Left Leaning Violation**.
 - **Rotate left** the appropriate node to fix.
- If there are two consecutive left links, we have an **Incorrect 4 Node Violation**.
 - **Rotate right** the appropriate node to fix.
- If there are any nodes with two red children, we have a **Temporary 4 Node**.
 - **Color flip** the node to emulate the split operation.

17.4 Runtime Analysis

Because a left-leaning red-black tree has a 1-1 correspondence with a 2-3 tree and will always remain within 2x the height of its 2-3 tree, the runtimes of the operations will take $\log(N)$ time.

Here's the abstracted code for insertion into a LLRB:

```
private Node put(Node h, Key key, Value val) {
    if (h == null) { return new Node(key, val, RED); }

    int cmp = key.compareTo(h.key);
    if (cmp < 0) { h.left = put(h.left, key, val); }
    else if (cmp > 0) { h.right = put(h.right, key, val); }
    else { h.val = val; }

    if (isRed(h.right) && !isRed(h.left)) { h = rotateLeft(h); }
    if (isRed(h.left) && isRed(h.left.left)) { h = rotateRight(h); }
    if (isRed(h.left) && isRed(h.right)) { flipColors(h); }

    return h;
}
```

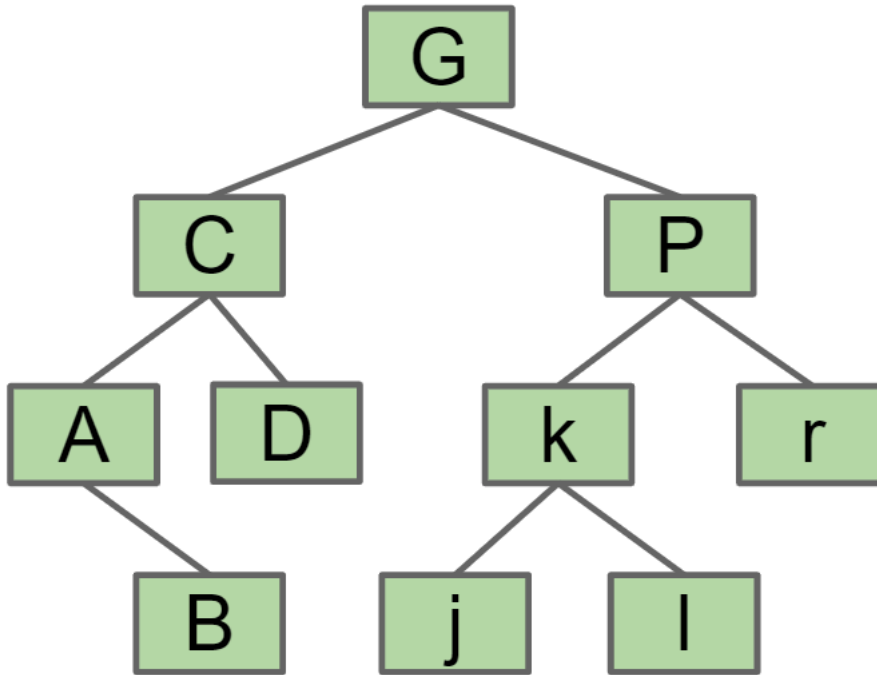
17.5 Summary

- Binary search trees are simple, but they are subject to imbalance which leads to crappy runtime. 2-3 Trees (B Trees) are balanced, but painful to implement.
- LLRB insertion is simple to implement (deletion is a bit harder to implement, we won't go over the specifics in this course).
 - Use three basic operations to maintain the balanced structure, namely rotateLeft, rotateRight, and color flip.
- LLRBs maintain correspondence with 2-3 trees, Standard Red-Black trees maintain correspondence with 2-3-4 trees.
 - Java's [TreeMap](#) is a red-black tree that corresponds to 2-3-4 trees.
 - 2-3-4 trees allow glue links on either side (see [Red-Black Tree](#)).
 - More complex implementation, but faster.

17.6 Exercises

Factual

1. Consider the tree below. If we call rotateLeft(C), which operation reverts the tree back to its original form?



2. When you rotate a nodes in a tree, which of the following can happen?
 - ☐ If the tree was previously a valid search tree, it can become invalid.
 - ☐ The height can stay the same.
 - ☐ The height can increase.
 - ☐ The height can decrease.
 - ☐ The number of nodes can change.
 - ☐ The root of the tree can change.

▼ Problem 1

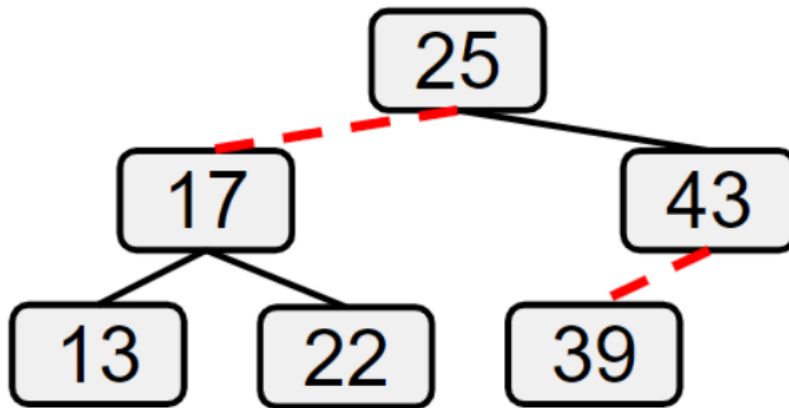
rotateRight(D). The inverse of any rotateLeft operation is a rotateRight on the node's left child.

▼ Problem 2

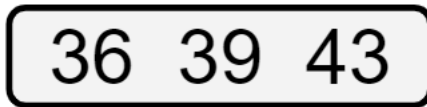
- ☐ If the tree was previously a valid search tree, it can become invalid. Rotation always preserves search tree properties.
- ☒ The height can stay the same. If the rotation does not affect any leaf nodes on the bottommost level, the height will stay the same.
- ☒ The height can increase. If the rotation affects a leaf on the bottommost level, the height can increase.
- ☒ The height can decrease. If the rotation affects a leaf on the bottommost level, the height can increase.
- ☐ The number of nodes can change. Rotation only changes the structure of the tree, not the nodes.
- ☒ The root of the tree can change. For example, rotating the root.

Procedural

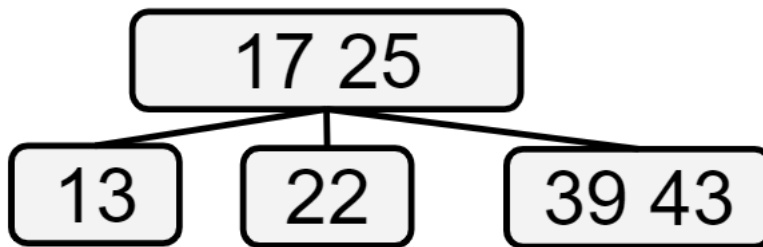
1. Consider the following LLRB. What is the height of the corresponding 2-3 tree and how many 3-nodes does it have?



2. Suppose we insert 15 in the LLRB above. What is the first operation that must be applied to maintain the LLRB invariants?
3. Suppose in the process of insertion, we end up with the following temporary 4-node. What is the corresponding LLRB representation of this node?



▼ Problem 1

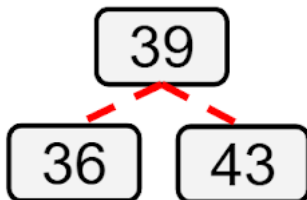


The corresponding 2-3 tree has height 1 and has two 3-nodes (17 25 and 39 43).

▼ Problem 2

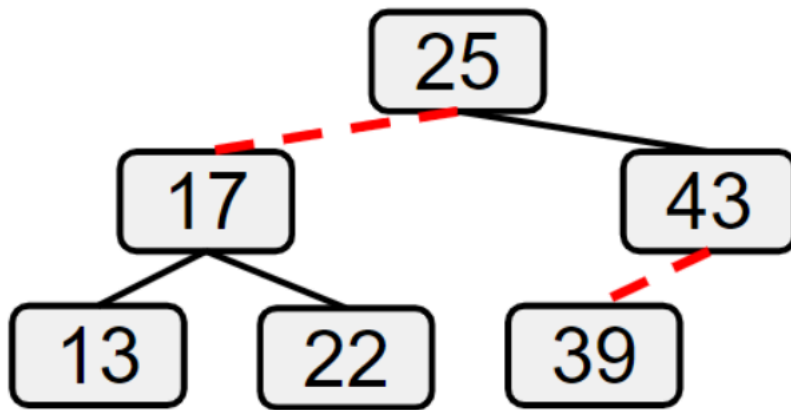
15 is inserted to the right of 13. Since we cannot have a right-leaning red link, we must rotateLeft(13).

▼ Problem 3



Metacognitive

1. Give a range of values, when inserted into the LLRB below, results in a rotateRight operation as the first balancing operation. Assume that values are distinct, but not necessarily integers.



▼ Problem 1

`rotateRight` occurs when we have two red links in a row. This occurs when we insert to the left of 39. This value must be larger than 25 (since it is in its right branch) but less than 39 (since it is the left child of 39). So our final range is (25, 39).

17. Red Black Trees

This chapter will continue our discussion on self-balancing trees through a more colorful lens.